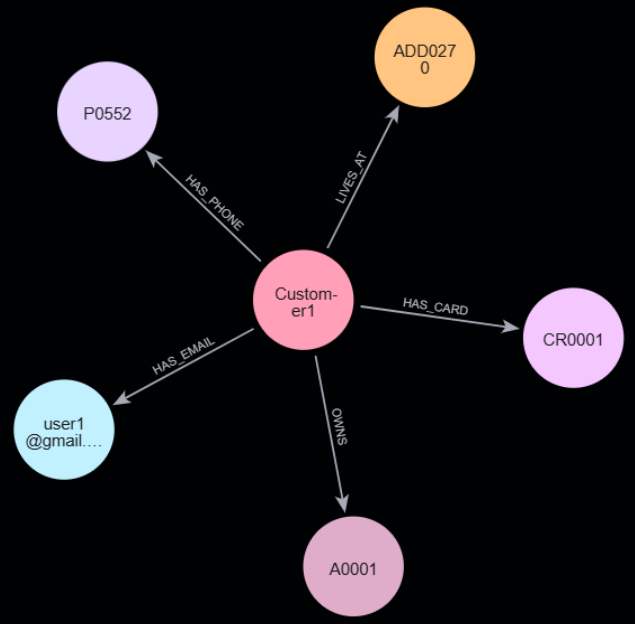
CUSTOMER 360 USE CASE USING NEO4J



Create a unified view of customer information by connecting customers, addresses, products, orders, and interactions in a graph database.

PROBLEM STATEMENT

The objective of this PoC was to demonstrate how Neo4j can be used to build a Customer 360 view and discover hidden relationships between customers.

In traditional relational databases, identifying relationships between customers often requires multiple joins across several tables. As the number of

entities and relationships increases, queries become more complex and difficult to visualize.

The goal was to evaluate whether a graph database could provide a simpler and more intuitive approach for:

* Customer 360 analysis
* Household identification
* Shared contact detection
* Relationship discovery
* Fraud analytics

USECASE DETAILS

Data Source

1) Sample datasets stored in GitHub.

2) Data imported into Neo4j using Cypher queries

Example:

```
LOAD CSV WITH HEADERS
FROM 'github_link'
AS row
CREATE (:Customer{
    customerId: row.Customer_ID,
    name: row.Name,
    dob: row.DOB,
    pan: row.PAN,
    cif: row.CIF
});
```

3) Nodes and relationships created to model customer connections

Example:

1. Customer → Account (OWNS)

```
LOAD CSV WITH HEADERS
FROM 'github_link'
AS row
MATCH (c:Customer {customerId: row.Customer_ID})
MATCH (a:Account {accountId: row.Account_ID})
CREATE (c)-[:OWNS]->(a);
```

In relational databases, relationships are calculated through joins.

In Neo4j, relationships are stored directly.

This allows Neo4j to navigate connections quickly and naturally.

NEO4J FEATURES & COMPARISON

1) Native Graph Storage: Data is stored as Nodes and Relationships, Relationships are stored directly, making traversals very fast.

2) High Performance Relationship Traversal: Traversing millions of relationships is much faster than complex SQL joins.

3) Why use graph database: Easy to traverse data, for deep nested data retrival. Can use graph algorithims

Neo4j vs Relational Database

Feature	Neo4j	SQL Database
Data Model	Graph	Tables
Relationships	Directly Stored	Joins Required
Complex Relationship Queries	Very Fast	Expensive Joins
Visualization	Built-in Graph View	External Tools Needed

COMPARE SQL VS NEO4J

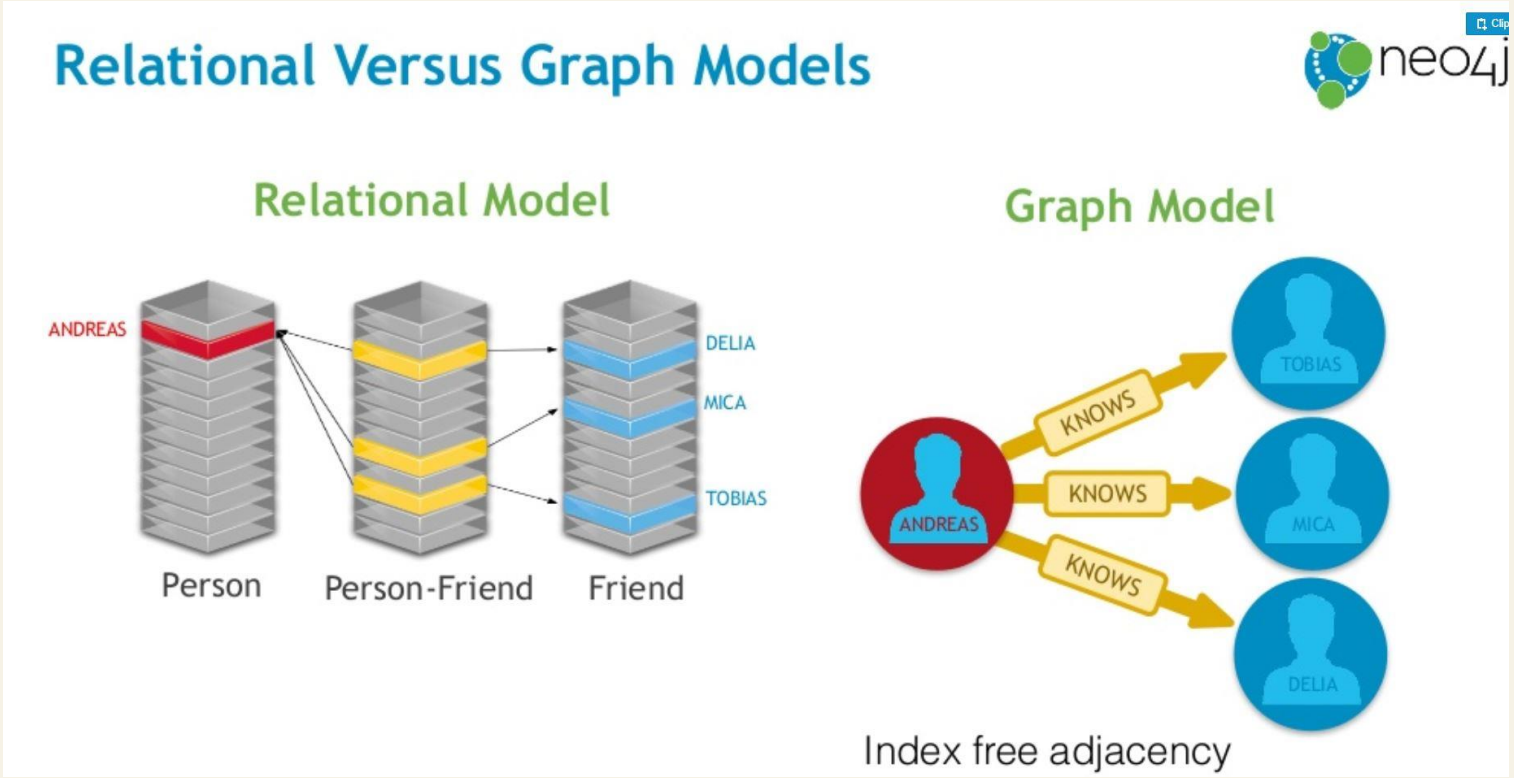
Example:

SQL

```
SELECT * FROM Customer c
JOIN Account a ON ...
JOIN Customer_Address ca ON ...
JOIN Address ad ON ...
JOIN Customer_Phone cp ON ...
JOIN Phone p ON ...
```

Neo4j

```
MATCH (c:Customer {customerId:'C001'})-[*1..2]-(n)
RETURN c,n
```



To answer questions such as:

* Who shares an address?

* Who shares a phone number?

* Which customers are connected indirectly?

Multiple table joins would be required using sql.In a relational database, answering questions about shared addresses, shared phone numbers, or indirect customer relationships requires multiple joins across many tables. As the network grows, those joins become increasingly complex. Neo4j stores relationships directly, so queries become simpler, more intuitive, and faster for connected-data use cases like Customer 360, fraud detection, and relationship discovery.

COMPLETE CUSTOMER 360

MATCH (c:Customer {customerId:'C001'})

OPTIONAL MATCH (c)-[r]-(n)

RETURN c,r,n

First, Neo4j finds the customer whose ID is C001. MATCH (c:Customer {customerId:'C001'})

Suppose that's John Smith.

Then: OPTIONAL MATCH (c)-[r]-(n) means: Find every node connected to John Smith and every relationship connected to him.

Finally:

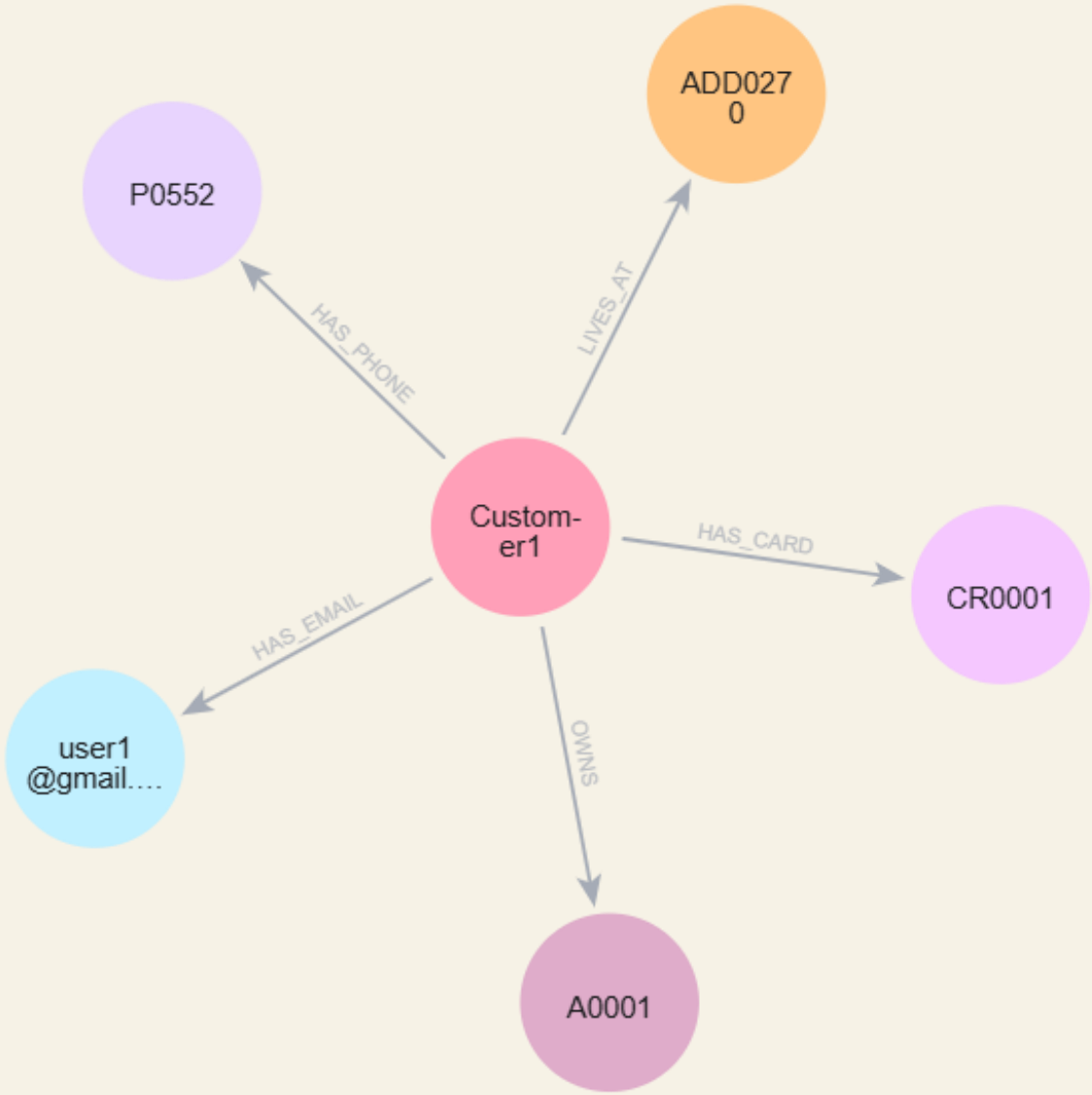
RETURN c,r,n returns the customer, relationship, and connected node.

Business Value

This gives a complete Customer 360 view from a single query.

This query starts with a customer and traverses all directly connected information such as accounts, cards, addresses, phones, and emails. Instead of joining

multiple tables, Neo4j follows relationships directly.


ADD0270
P0552
HAS_PHONE
LIVES_AT
Custom-er1
HAS_CARD
CR0001
HAS_EMAIL
user1@gmail....
OWNS
A0001

CUSTOMERS SHARING THE SAME ADDRESS

MATCH (c1:Customer)-[:LIVES_AT]->(a:Address)<-[:LIVES_AT]-(c2:Customer)

WHERE c1.customerId <> c2.customerId

RETURN c1.name,c2.name,a.addressLine

Example Suppose:

John Smith -> ADD001

Mary Smith -> ADD001

Neo4j finds:

John Smith <-- ADD001 --> Mary Smith

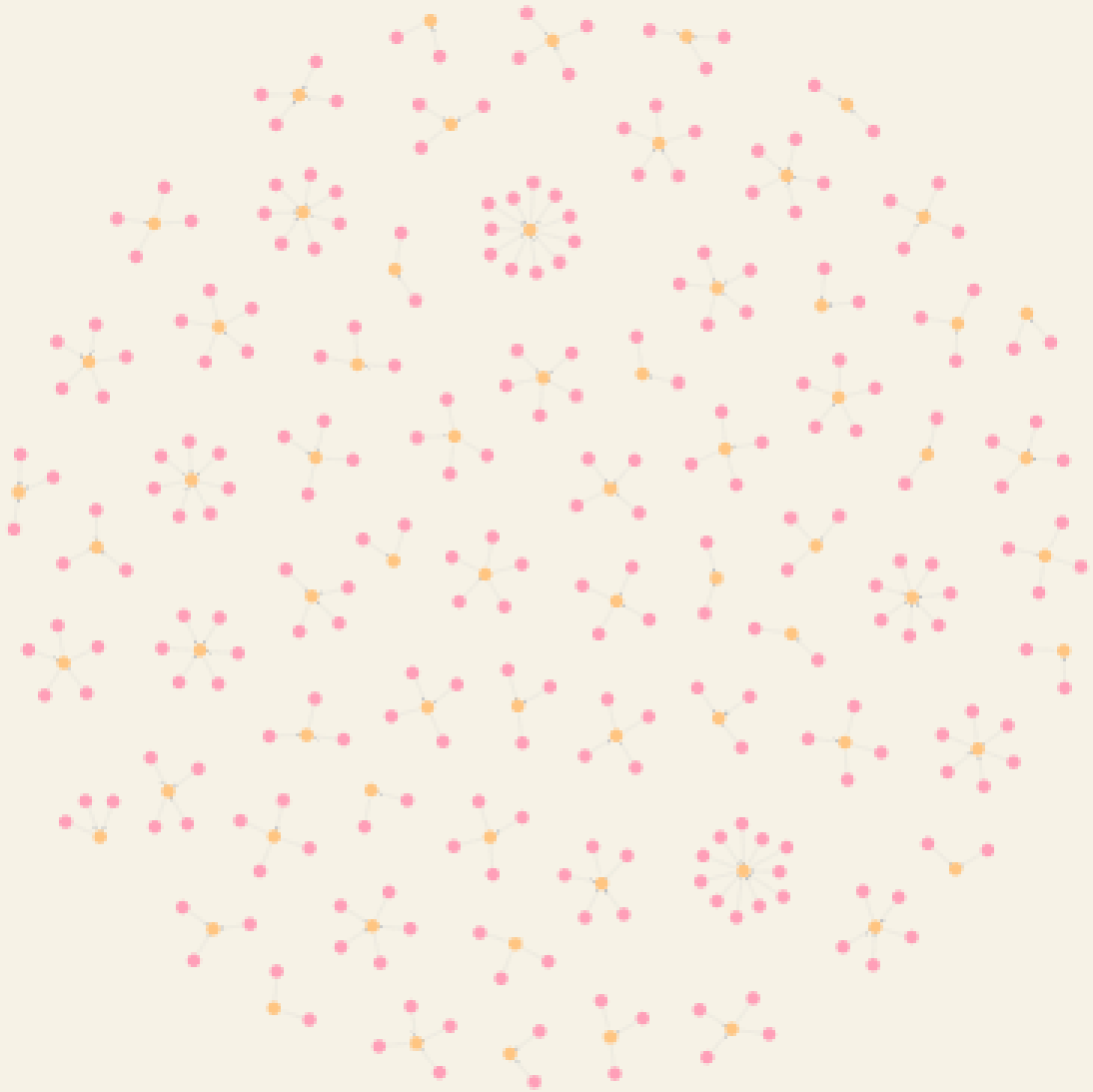
Why WHERE clause? WHERE c1.customerId <> c2.customerId

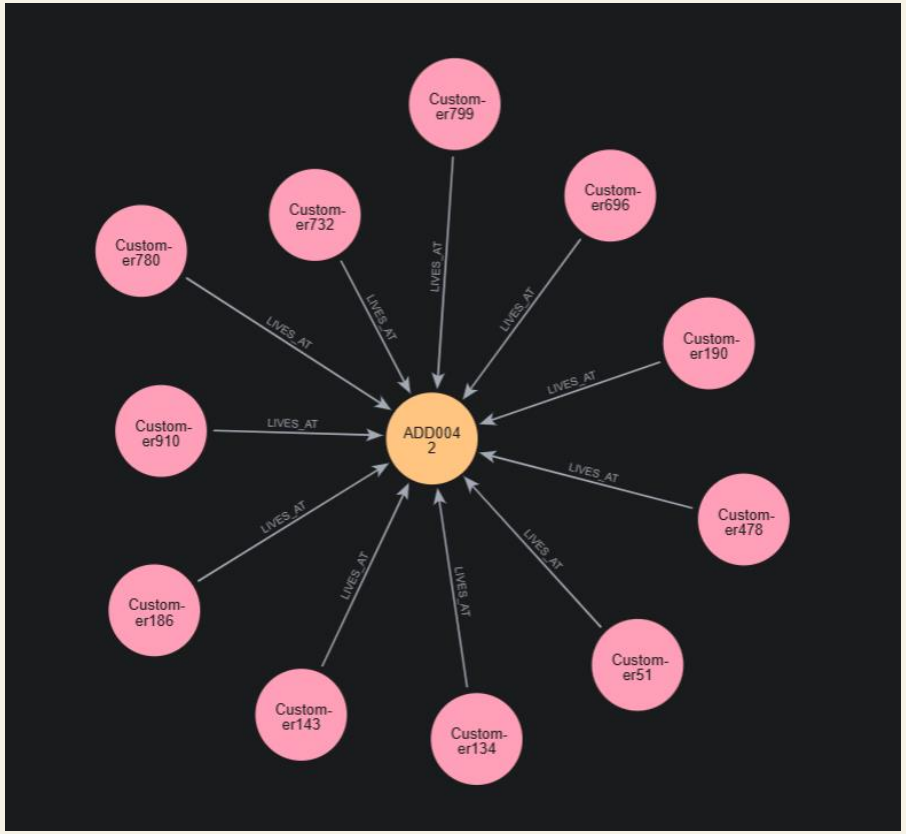
Without this condition: John Smith = John Smith would also appear.

We only want different customers.

Business Value : Household identification, Family grouping, Relationship discovery, Fraud investigation

This query identifies customers linked to the same address. In banking, this can help identify households, family members, or potentially suspicious relationships.




Custom-er799
Custom-er732
Custom-er696
Custom-er780
Custom-er190
Custom-er910
ADD0042
Custom-er478
Custom-er186
Custom-er51
Custom-er143
Custom-er134
LIVES_AT

CUSTOMERS SHARING PHONE NUMBER

MATCH (c1:Customer)-[:HAS_PHONE]->(p:Phone)<-[:HAS_PHONE]-(c2:Customer)

WHERE c1 <> c2

RETURN c1,c2,p

Example

John -> 9999999999

Mark -> 9999999999

Graph:

John

\

9999999999

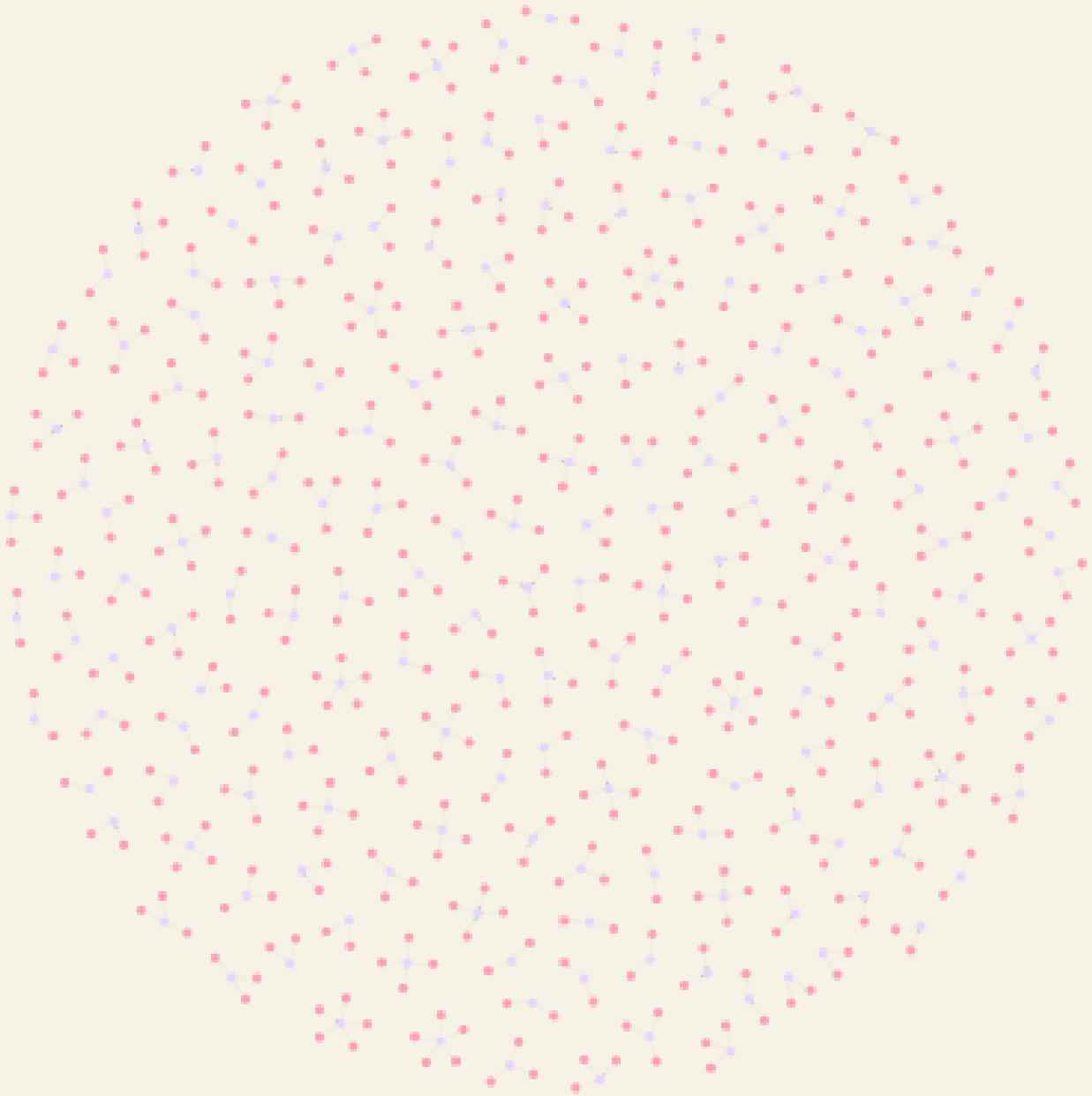
/

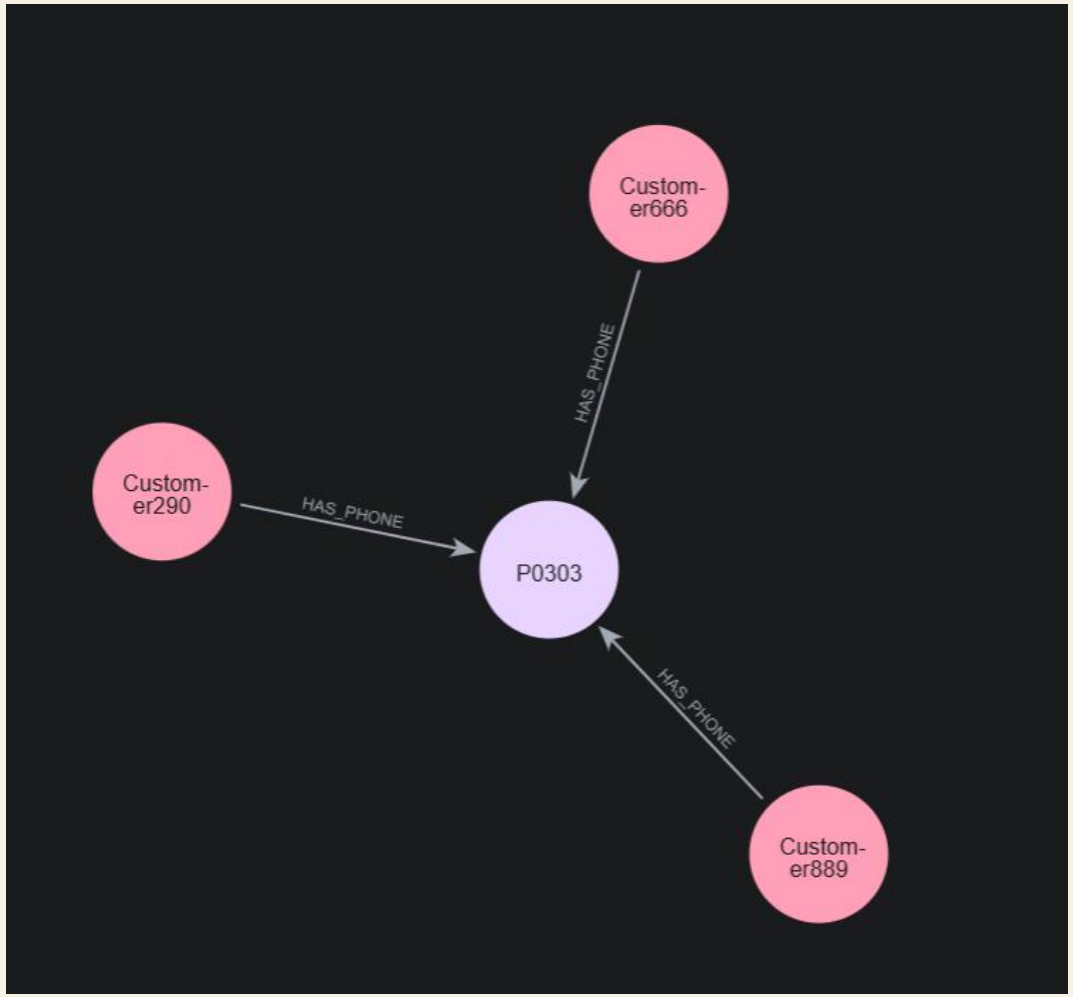
Mark

Neo4j immediately finds this relationship.

Business Value: Synthetic identity detection, Mule account detection, Fraud analysis

This query identifies customers sharing the same phone number. Such relationships may indicate legitimate family connections or potentially suspicious activity.




Custom-
er666
HAS_PHONE
Custom-
er290
HAS_PHONE
P0303
HAS_PHONE
Custom-
er889

RELATED CUSTOMERS WITHIN 3 HOPS

```
MATCH p=(c:Customer {customerId:'C001'})-[*1..3]-(related)
RETURN p
```

This demonstrates graph traversal.

What does *1..3 mean?

[*1..3] means: Find anything connected within:

1 hop, 2 hops, 3 hops

John -> Address -> Mar (2 hops)

John -> Address -> Mary -> Phone (3 hops)

In SQL:

Customer JOIN Address JOIN Customer JOIN Phone

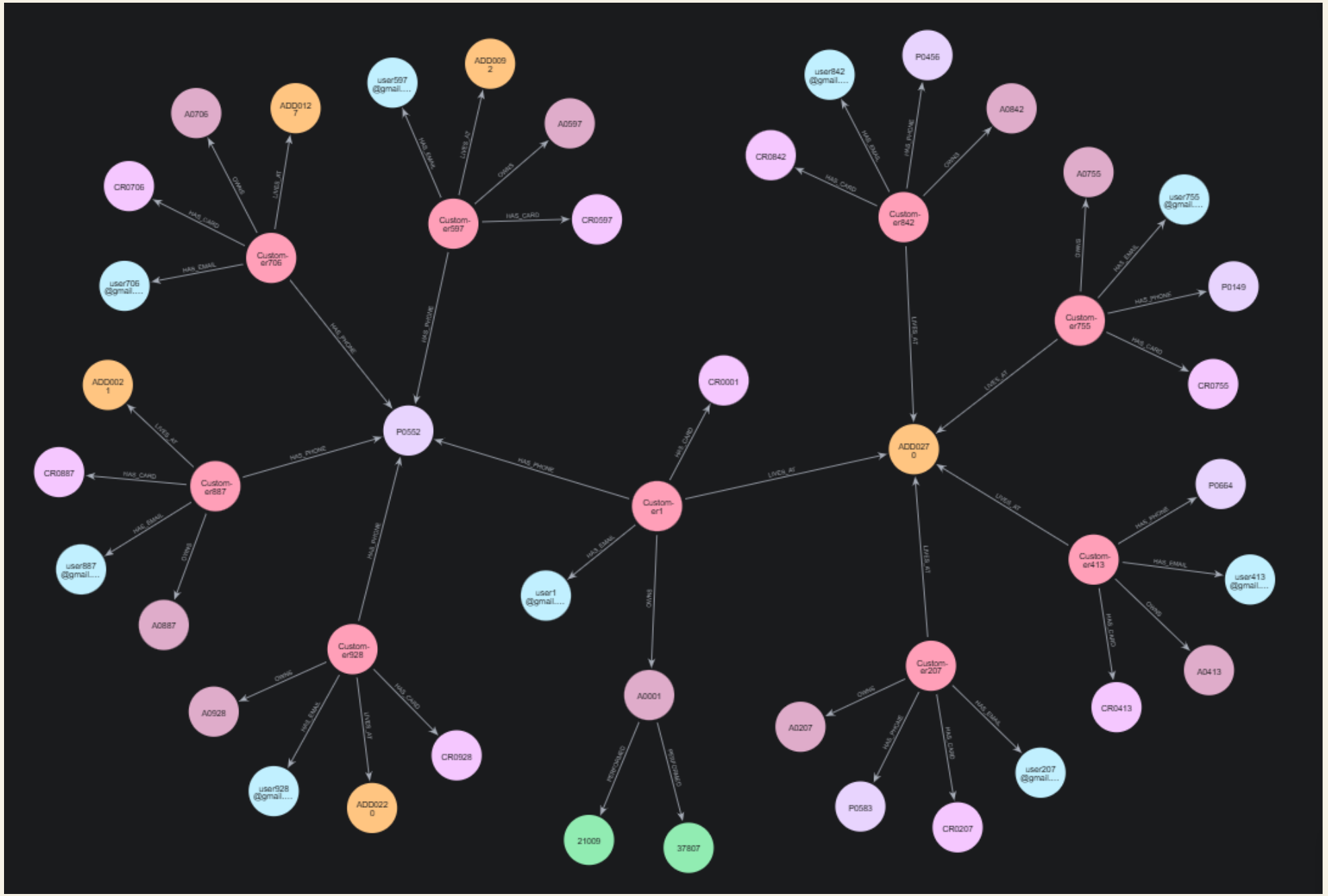
multiple joins are needed.

In Neo4j:

[*1..3] handles traversal naturally.

Business Value Network analysis, Relationship discovery, Fraud rings, Connected customer groups

This query explores the customer's relationship network up to three levels deep. It helps uncover indirect connections that may not be visible through traditional reporting.


A0706
ADD0127
CR0706
Custom-er706
user706@gmail....
user597@gmail....
ADD0092
A0597
Custom-er597
CR0597
ADD0021
CR0887
Custom-er887
user887@gmail....
A0887
P0552
CR0001
Custom-er1
user1@gmail....
A0001
21009
37807
Custom-er928
A0928
user928@gmail....
ADD0220
CR0928
user842@gmail....
P0456
A0842
CR0842
Custom-er842
A0755
user755@gmail....
P0149
Custom-er755
CR0755
ADD0270
P0664
Custom-er413
user413@gmail....
A0413
CR0413
Custom-er207
A0207
P0583
CR0207
user207@gmail....
OWNS
HAS_CARD
HAS_EMAIL
HAS_PHONE
LIVES_AT

MOST CONNECTED CUSTOMERS

MATCH (c:Customer)

RETURN c.name,size((c)--()) as connections

ORDER BY connections DESC

What it does

Find all customers: MATCH (c:Customer)

Count their connections: size((c)--())

Example Output

Customer Connections

John Smith 5

Mary Smith 4

David Jones 3

Business Value Influential customers, Highly connected entities, Fraud hotspot analysis

This query ranks customers based on the number of relationships they have in the graph. Highly connected customers often become important investigation points

in customer analytics and fraud detection.

c.name	connections
Customer1	5
Customer2	5
Customer3	5
Customer4	5
Customer5	5
Customer6	5
Customer7	5
Customer8	5
Customer9	5
Customer10	5

Showing **1-10** of **1000** results

THANK YOU